HR Policy Assistant Using LangGraph and RAG

Capstone Project Documentation

Submitted by

Name: **Preetam Mondal**

Roll No.: **23051768**

Program: **B.Tech (CSE)**

Batch: **2023 – 2027**

Course: **Agentic AI**

Instructor: **Dr. Kanthi Kiran Sirra**

1. Problem Statement

In large organisations, employees frequently depend on the Human Resources (HR) department to resolve routine policy-related queries. Questions regarding leave entitlements, attendance regulations, salary structure, resignation procedures, and work-from-home eligibility are among the most common, yet repetitive, requests handled by HR teams on a daily basis.

Traditional HR support systems suffer from several inherent inefficiencies. First, HR personnel are required to respond manually to queries that could be addressed through documented policies, resulting in significant time and resource wastage. Second, the absence of a 24/7 automated assistance mechanism means that employees cannot obtain timely responses outside of business hours. Third, verbal or informal communication of policies introduces inconsistency and the risk of misinformation.

These challenges collectively reduce organisational productivity, increase HR overhead, and lead to employee dissatisfaction. There is, therefore, a pressing need for an intelligent, automated system capable of answering HR policy queries accurately, consistently, and at any time of day.

2. Proposed Solution

This project proposes an AI-powered HR Policy Assistant that leverages Retrieval-Augmented Generation (RAG) and a LangGraph-based agent workflow to provide accurate, context-grounded responses to employee queries. The system ingests ten structured HR policy documents covering Leave, Payroll, Attendance, Insurance, Employee Benefits, Resignation, Code of Conduct, Promotion, Grievance Handling, and Work From Home policies.

The proposed solution is built around three principal design goals:

- Accuracy: Responses are generated exclusively from the retrieved context, with an explicit no-hallucination constraint enforced through the system prompt.
- Automation: An intelligent routing mechanism determines, for each query, whether to retrieve from the knowledge base, invoke a real-time tool, or respond from conversational memory.
- Reduced HR Workload: By handling repetitive policy queries autonomously, the system frees HR personnel to focus on complex, high-value tasks.

The assistant is deployed as an interactive Streamlit web application, providing employees with a familiar chat interface accessible at any time.

3. System Architecture and Workflow

The agent pipeline is implemented as a directed acyclic graph (DAG) using the LangGraph StateGraph API. The graph operates on a shared state object of type CapstoneState, which carries all intermediate data across nodes. The pipeline consists of eight nodes, each responsible for a discrete function.

3.1 State Definition

The CapstoneState TypedDict defines the schema for the shared state and includes the following fields: question (the user's input), messages (conversation history), route (routing decision), retrieved (context fetched from ChromaDB), sources (list of retrieved topic labels), tool_result (output from the date/time tool), answer (the generated response), faithfulness (evaluation score), eval_retries (retry counter), and user_name (personalisation field extracted from conversation).

3.2 Memory Node

The Memory Node is the entry point of the graph. It appends the incoming user query to the conversation history list and retains only the last six messages to manage the context window. Additionally, it performs a simple natural language extraction: if the query contains the phrase 'my name is', the node extracts and stores the user's name in the user_name state field, enabling personalised responses throughout the session.

3.3 Router Node

The Router Node classifies each query into one of three routing categories: retrieve, tool, or skip. A rule-based override is applied first for efficiency and reliability — queries containing the keywords 'date' or 'time' are immediately routed to the Tool Node, while queries involving 'my name' or 'who am i' are routed to the Skip Node. For all other queries, the Groq-hosted LLaMA-3.3-70B model is invoked with a classification prompt that instructs it to return exactly one of the three route labels. If the model returns an unrecognised value, the route defaults to 'retrieve' as a safe fallback.

3.4 Retrieval Node

When the route is 'retrieve', the Retrieval Node queries the ChromaDB vector store. The user's question is first encoded into a dense vector using the all-MiniLM-L6-v2 SentenceTransformer model. The top three semantically similar documents are retrieved from the 'hr_kb' collection along with their metadata. The retrieved document texts are concatenated with their topic labels into a formatted context string that is passed to the Answer Node.

3.5 Skip Node

The Skip Node handles queries that are memory-based or personal in nature (e.g., recalling the user's name). It sets the retrieved context to an empty string and the sources list to empty, signalling to the Answer Node that no RAG context is available and that the response should be drawn from the conversation memory held in state.

3.6 Tool Node

The Tool Node provides real-time date information. When invoked, it calls Python's datetime module to retrieve the current date formatted as YYYY-MM-DD, and stores the result in the tool_result state field. This node demonstrates the system's ability to integrate live, non-retrieval data sources within the agentic pipeline.

3.7 Answer Node

The Answer Node is responsible for generating the final response using the Groq-hosted LLaMA-3.3-70B model. It follows a structured decision flow with specialised handling before invoking the LLM.

- name recall queries such as “What is my name?” or “Who am I?” are handled directly from the stored user_name in state, without invoking the LLM.
- emotional inputs containing stress-related keywords trigger a predefined empathetic response directing the user to HR or a counsellor.

For all other queries, the node constructs a structured system prompt incorporating retrieved context, tool output, and recent conversation history. The prompt enforces strict constraints: responses must be grounded only in the provided context or tool output, and if the answer is not available, the system must reply with “I don’t know. Please contact HR.”

If a user_name is available, it is prefixed to the response to maintain conversational personalisation, ensuring consistent and user-aware interactions.

3.8 Evaluation Node

The Evaluation Node implements a self-assessment loop for RAG-based responses. It invokes the LLM with a compact prompt asking it to rate the faithfulness of the generated answer to the retrieved context on a scale of 0 to 1. If the faithfulness score falls below 0.7 and the retry count is less than 2, the graph routes back to the Answer Node for regeneration. For tool-only or memory-only responses (where context is empty), faithfulness is set to 1.0 and evaluation is bypassed.

3.9 Save Node and Graph Compilation

The Save Node appends the final answer to the messages list, thereby persisting it as part of the conversation history for subsequent turns. The graph is compiled using graph.compile(checkpointer=MemorySaver()), which attaches an in-memory checkpointer that maintains state across invocations within the same thread_id, enabling true multi-turn conversation memory.

4. Technologies Used

4.1 Python

Python 3 serves as the primary programming language for all components of this project, including the agent pipeline, embedding generation, vector store operations, and the Streamlit user interface.

4.2 LangGraph

LangGraph is a framework built on top of LangChain that enables the construction of stateful multi-actor agent graphs. In this project, it is used to define the eight-node directed graph with conditional and unconditional edges, manage shared state via CapstoneState, and compile the graph with a persistent memory checkpointer.

4.3 LangChain and Groq LLM

LangChain's ChatGroq integration is used to interface with the Groq inference API, which hosts the LLaMA-3.3-70B-Versatile language model. This model is responsible for query classification, answer generation, and faithfulness evaluation. The model is initialised with a maximum of three retries to handle transient API failures.

4.4 ChromaDB

ChromaDB is an open-source vector database used to store and retrieve HR policy document embeddings. A single in-memory client is initialised, and a collection named 'hr_kb' is created or retrieved at startup. Documents are stored with their text, embedding vectors, unique IDs, and topic metadata.

4.5 Sentence Transformers

The all-MiniLM-L6-v2 model from the Sentence Transformers library is used to generate dense vector embeddings for both the HR policy documents (at load time) and user queries (at retrieval time). This lightweight model provides an effective balance of embedding quality and inference speed.

4.6 Streamlit

Streamlit provides the interactive web-based chat interface. The application uses Streamlit's session state to persist chat history and thread identifiers across user interactions. A sidebar provides contextual information about the system and a 'New Conversation' button that resets the session and generates a fresh thread_id.

5. Features of the System

- Multi-Route Agent: The system dynamically routes each query to one of three paths — retrieval (RAG), tool invocation (date/time), or memory skip — based on rule-based overrides followed by LLM classification.
- Memory-Based Personalisation: The MemorySaver checkpointer and the memory_node together enable persistent, multi-turn conversations. User names introduced in one turn are recalled in subsequent turns, and responses are personalised accordingly.
- RAG-Based Factual Answering: All policy-related queries are answered exclusively from documents retrieved from the ChromaDB vector store, ensuring grounded and accurate responses.
- Self-Evaluation Loop: Following each RAG-based answer, the Evaluation Node scores faithfulness using the LLM itself. Answers scoring below 0.7 are regenerated, with a maximum of two retry attempts to prevent infinite loops.
- Streamlit Interactive UI: The front-end provides a full-featured chat interface with conversation history display, a sidebar with usage information, and a session reset button.
- No-Hallucination Policy: The system prompt explicitly instructs the model not to introduce information beyond what is present in the retrieved context or tool output, with a defined fallback message for unanswerable queries.
- Emotional Intelligence: A simple keyword-based filter detects stress-related language and returns a compassionate predefined response directing the employee to HR or a counsellor.

6. Implementation Details

6.1 Knowledge Base Construction

Ten HR policy documents are defined inline in agent.py, each represented as a Python dictionary with three keys: id (e.g., 'doc_001'), topic (e.g., 'Leave Policy'), and text (the full policy content). The load_knowledge_base() function extracts the text, IDs, and topic metadata from these dictionaries, generates embeddings for all ten documents in a single batch using embedder.encode(), and adds them to the ChromaDB collection via collection.add(). An existence check (collection.count() == 0) prevents duplicate insertion on repeated application restarts.

6.2 Embedding Generation

All embeddings are produced using the all-MiniLM-L6-v2 SentenceTransformer model, which generates 384-dimensional dense vectors. At query time, the user's question is encoded using the same model, and the resulting vector is passed to collection.query() to perform approximate nearest-neighbour search. The top three results are returned along with their document texts and metadata.

6.3 Routing Logic

The router_node function implements a two-stage classification strategy. In the first stage, a deterministic keyword check is applied: the presence of 'date' or 'time' maps to the 'tool' route, while 'my name' or 'who am i' maps to 'skip'. In the second stage, for all remaining queries, the LLM is prompted with a three-class classification task. The raw model output is converted to lowercase and validated against the set {'retrieve', 'tool', 'skip'}; any unrecognised output defaults to 'retrieve'.

6.4 Prompt Design

The Answer Node constructs a structured system prompt that embeds four dynamic components: the retrieved context (formatted with topic labels), the tool output (empty string if not applicable), the conversation history (last six messages), and the retry count. The prompt enforces four explicit rules: prioritise tool output over context when both are present; answer only from context when tool output is absent; use topic labels to locate relevant content; and respond with 'I don't know. Please contact HR.' when the answer cannot be found. This design minimises hallucination and ensures consistent, traceable responses.

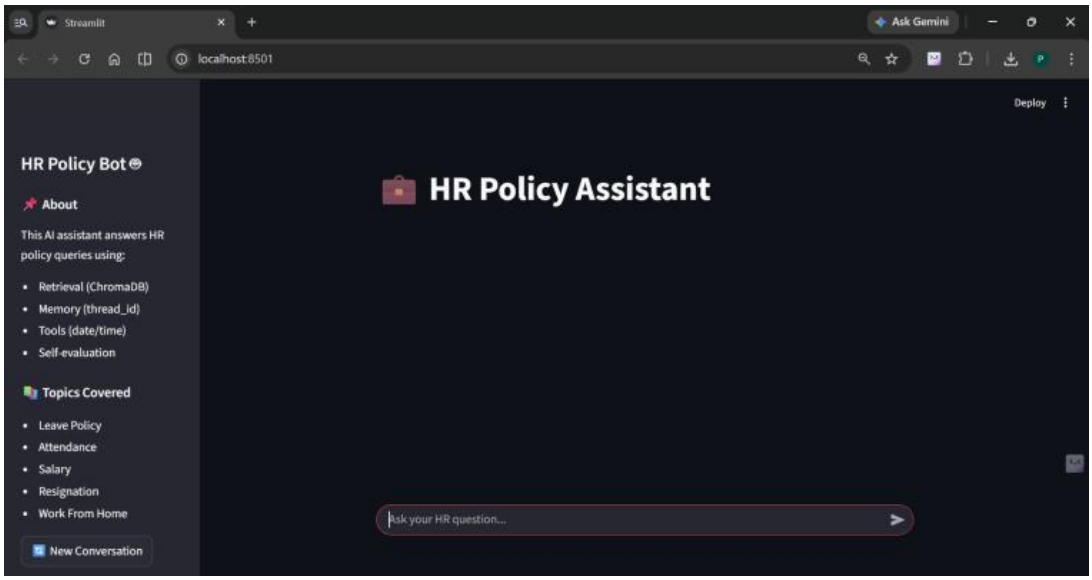
6.5 API Key Security

The Groq API key is not hard-coded in the source code. Instead, it is accessed securely at runtime using environment variables via os.getenv('GROQ_API_KEY'). This approach ensures that sensitive credentials are not exposed in the codebase or version control systems such as GitHub, thereby maintaining security and preventing accidental leaks.

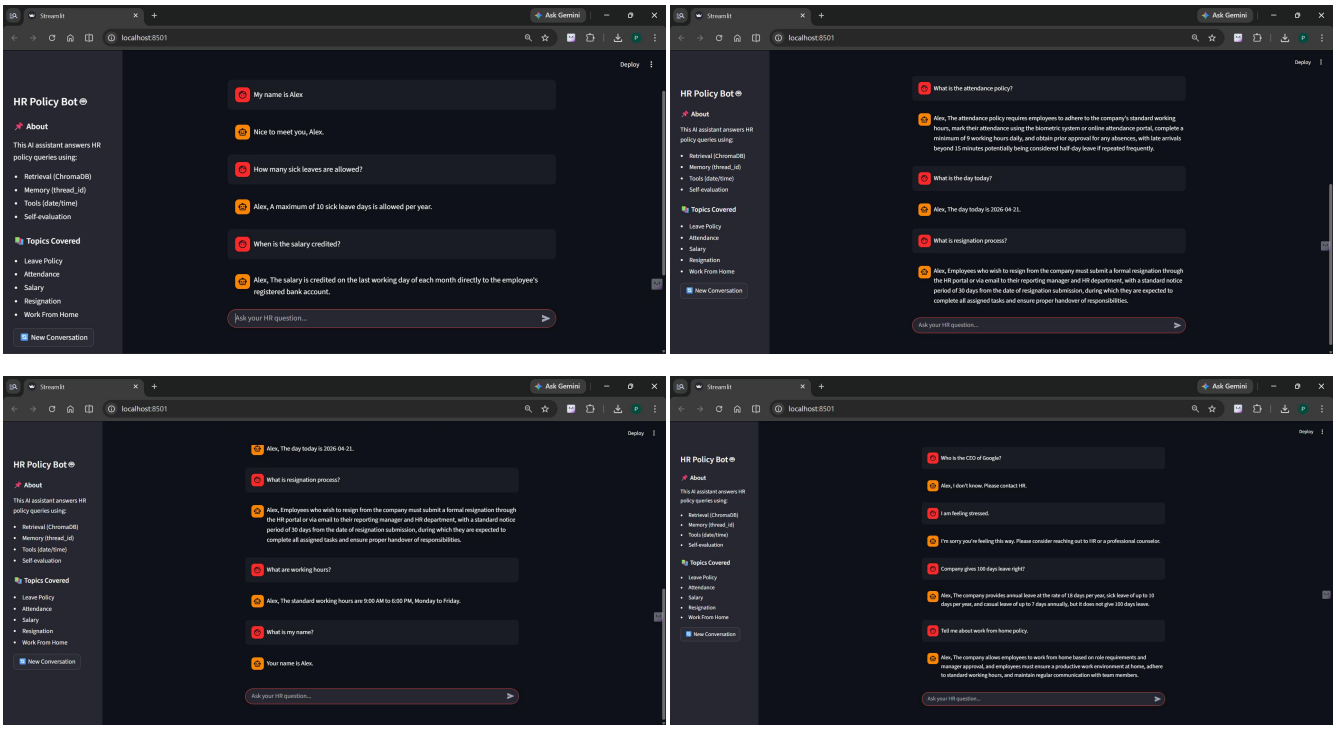
7. Screenshots

The following figures illustrate key aspects of the system's user interface and output.

Figure 1: Chat Interface — Overview of the Streamlit chat UI with the HR Policy Bot sidebar, chat history display area, and query input field.



Figures 2-5: Query Response Examples — illustrate the system's behaviour across different query types, including memory-based personalisation, policy retrieval, tool-based responses, and fallback handling. The assistant maintains conversational context, delivers accurate HR policy information, responds appropriately to emotional inputs, and ensures consistent behaviour across multi-turn interactions.



8. Evaluation and Results

The system was evaluated across three dimensions during development using the accompanying Jupyter notebook: faithfulness of RAG-based answers, correctness of routing decisions, and proper fallback behaviour for out-of-scope queries.

8.1 Faithfulness Evaluation

- The system's faithfulness was evaluated using the Evaluation Node, which scores responses on a scale of 0 to 1 based on their alignment with the retrieved context.
- In testing, most policy-related queries achieved scores between 0.85 and 1.0, indicating high alignment with source documents. The retry mechanism was triggered only in a few cases, demonstrating the effectiveness of the prompt design.
- Out-of-scope queries consistently triggered the fallback response "I don't know. Please contact HR.", ensuring that no unsupported or hallucinated information was generated.

8.2 Routing Accuracy

The routing mechanism performed reliably across different query types. Rule-based overrides successfully handled date/time queries and memory-related inputs without requiring LLM classification. Policy-related queries were generally routed to the retrieval path, while tool-based and memory-based queries were correctly directed to their respective nodes. This hybrid routing strategy ensured both efficiency and accuracy in handling diverse user inputs.

8.3 Fallback Handling

Queries outside the scope of the ten HR policy documents (e.g., technical IT questions) were correctly handled: the retrieval returned low-relevance context, and the LLM adhered to the system prompt's instruction to respond with 'I don't know. Please contact HR.' rather than fabricating an answer.

8.4 Memory and Personalisation

Multi-turn conversation tests confirmed that the MemorySaver checkpointer successfully preserved the user_name field across turns within the same thread_id. The system correctly recalled names introduced earlier and prepended them to responses throughout the session.

9. Distinguishing Features

- LangGraph Modular Design: The use of LangGraph's StateGraph allows each processing concern (memory, routing, retrieval, tool use, generation, evaluation, persistence) to be encapsulated in an independent node, making the pipeline highly maintainable and extensible.
- Self-Evaluation with Retry: The faithfulness scoring loop, implemented entirely within the agent graph, enables the system to autonomously detect and correct low-quality responses without external intervention.
- Hybrid Routing Strategy: The combination of deterministic keyword matching and LLM-based classification provides both speed and flexibility in handling diverse query types.
- No-Hallucination Constraint: The explicit 'I don't know. Please contact HR.' fallback, enforced via the system prompt, ensures that the assistant never invents information not present in its knowledge base.
- Thread-Based Persistent Memory: The MemorySaver checkpointer tied to unique thread_ids enables genuinely stateful, personalised conversations across multiple turns.

10. Limitations

- Limited Knowledge Base: The system's knowledge is restricted to ten manually authored policy documents. Queries touching on policies or scenarios not covered by these documents will result in fallback responses.
- No Multi-Hop Reasoning: The current implementation performs single-step retrieval. Complex queries requiring synthesis across multiple policy areas are not explicitly supported.
- In-Memory Vector Store: The system currently uses an in-memory ChromaDB instance, meaning that the vector database is not persisted to disk and must be reloaded each time the application starts. This limits scalability and long-term deployment capabilities.
- Simplistic Evaluation Metric: Faithfulness scoring relies on the same LLM used for generation, introducing potential bias. A dedicated external evaluation framework would provide more objective scoring.
- No Deployment Scalability: The application is designed for local or single-user deployment. It has not been configured for production-scale cloud hosting.

11. Future Improvements

- Larger and Dynamic Knowledge Base: Expanding the knowledge base to include all organisational HR documents, with support for dynamic document ingestion and a persistent ChromaDB disk-backed store.
- Multi-Modal Inputs: Enabling employees to upload scanned policy PDFs or images, with OCR-based text extraction feeding directly into the vector store.
- Cloud Deployment: Containerising the application using Docker and deploying it on a cloud platform (e.g., AWS, GCP, or Azure) to support concurrent multi-user access.
- Advanced Evaluation Framework: Integrating a dedicated RAG evaluation library such as RAGAS to compute faithfulness, answer relevancy, and context precision using independent, reference-based metrics.
- Improved UI/UX: Enhancing the Streamlit interface with features such as source citation display, conversation export, and topic-based quick-access buttons.
- Multi-Hop Reasoning: Implementing a query decomposition module to handle complex, multi-part HR queries that require information synthesis from more than one policy area.

12. Conclusion

This capstone project demonstrates the design and implementation of a functional, intelligent HR Policy Assistant using LangGraph, Retrieval-Augmented Generation, and a Groq-hosted large language model. The system addresses the practical problem of repetitive, manual HR query resolution by providing employees with an accurate, available, and personalised conversational interface.

The agent pipeline, structured as an eight-node LangGraph StateGraph, effectively separates concerns across memory management, intelligent routing, vector-based retrieval, real-time tool invocation, LLM-based generation, self-evaluation, and state persistence. The incorporation of a faithfulness-based retry loop represents a notable step towards self-correcting agentic systems, while the explicit no-hallucination constraint ensures trustworthy outputs within the defined knowledge domain.

From an academic perspective, this project integrates concepts from natural language processing, vector databases, graph-based agent design, and software engineering into a cohesive, deployable application. It serves as a practical demonstration of how modern agentic AI frameworks can be applied to solve real-world organisational challenges.